

Scaling Laws for Large Language Models

A Survey of Pretraining and Reinforcement-Learning Regimes

June 2026

Abstract

Scaling laws are empirical relationships that predict how a model’s loss or task performance changes as a function of resources — parameters, data, and compute. They have become the central planning tool of modern LLM development: they tell labs how to spend a fixed compute budget before training begins. This report reviews the two regimes that now govern frontier models. The first is *pretraining*, where the Kaplan and Chinchilla power laws, their compute-optimal allocation, and their many refinements (data-constrained, inference-aware, precision, mixture-of-experts) are mature and predictive. The second is *reinforcement learning* — both the RLHF alignment phase and the newer “RL-on-verifiable-rewards” regime behind reasoning models such as o1 and DeepSeek-R1 — where scaling is real but the laws are younger, noisier, and dominated by a distinctive train-time/test-time compute trade-off. We give the functional forms, the headline constants, the practical takeaways, and the main caveats throughout.

1 What a scaling law is, and why it matters

A neural scaling law is an empirical claim that some performance metric — almost always cross-entropy loss $\mathcal{L}$ on held-out text — falls as a smooth *power law* in a resource x (model size N , dataset size D , or compute C) over many orders of magnitude:

$$\mathcal{L}(x) \approx \mathcal{L}_\infty + \left(\frac{x_0}{x}\right)^\alpha.$$

Two properties make this useful. First, **predictability**: a curve fit on small, cheap runs extrapolates to the loss of a model too expensive to train more than once. Second, **compute allocation**: given a fixed budget C , the laws say how to split it between making the model *bigger* and training it on *more data*. The dominant cost accounting throughout is the rule of thumb

$$C \approx 6ND \quad (\text{FLOPs}), \tag{1}$$

i.e. training a dense transformer with N parameters on D tokens costs about $6ND$ floating-point operations (2 for the forward pass and 4 for the backward pass, per parameter per token).

2 Pretraining scaling laws

2.1 Kaplan et al. (2020): the original power laws

The foundational result [1] established that LM loss scales as a power law independently in N , D , and C when the other two are not bottlenecks:

$$\mathcal{L}(N) \propto N^{-\alpha_N}, \quad \mathcal{L}(D) \propto D^{-\alpha_D}, \quad \mathcal{L}(C) \propto C^{-\alpha_C},$$

with small exponents ($\alpha_N \approx 0.076$). Crucially, the laws were remarkably insensitive to architectural details (depth vs. width, attention shape) over the ranges studied — loss is governed mainly by scale, not by design choices. Kaplan’s analysis, however, recommended putting *most* of a larger compute budget into model size, training relatively few tokens. This allocation was later shown to be wrong.

2.2 Chinchilla (2022): compute-optimal training

Hoffmann et al. [2] re-examined the allocation question with more careful experiments (correcting issues such as a sub-optimal learning-rate schedule in Kaplan’s runs) and reached a different, now-standard conclusion: *for compute-optimal training, model size and data should be scaled in roughly equal proportion*. They fit the joint functional form

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2)$$

where E is the irreducible loss (entropy of natural text), and the two terms capture finite-model and finite-data error. The fitted constants were approximately

$$E \approx 1.69, \quad A \approx 406.4, \quad B \approx 410.7, \quad \alpha \approx 0.34, \quad \beta \approx 0.28.$$

Minimizing (2) under the constraint $C = 6ND$ gives the compute-optimal frontier

$$N_{\text{opt}} \propto C^a, \quad D_{\text{opt}} \propto C^b, \quad a \approx b \approx 0.5,$$

i.e. both grow as roughly the square root of compute. The practical distillation is the **“20 tokens per parameter”** heuristic: a compute-optimal model trains on about $D \approx 20N$ tokens. The 700M–70B Chinchilla model, trained on 1.4T tokens, outperformed the 4×-larger Gopher trained on fewer tokens — a direct demonstration that many earlier models were badly undertrained.

2.3 Reading the functional form

Equation (2) is more than a curve fit; each term is interpretable. E is a floor set by the data distribution. A/N^α is the penalty for a model with finite capacity, and B/D^β the penalty for having seen finite data. Because $\alpha > \beta$, returns to parameters fall off slightly faster than returns to data — part of why the optimal balance tilts data-ward relative to Kaplan. The form also makes explicit that you cannot buy your way out of a data wall with parameters alone: as $N \rightarrow \infty$ at fixed D , loss bottoms out at $E + B/D^\beta$.

2.4 Data-constrained scaling: repeating tokens

The Chinchilla recipe assumes unlimited fresh data. Frontier models now approach the supply limit of high-quality web text, raising the question of how to scale when tokens must be *repeated*. Muennighoff et al. [3] fit a modified law in which repeated tokens and extra parameters have exponentially *decaying* marginal value. The key findings: training for up to ~ 4 epochs incurs almost no penalty relative to fresh data; beyond roughly 4–16 epochs, repetition adds little and eventually nothing. They generalize (2) by replacing N and D with *effective* counts N' , D' that discount repeated data and excess parameters. This regime is increasingly the operative one for large models.

2.5 Inference-aware scaling: training past the optimum

Chinchilla minimizes *training* compute for a target loss. But a deployed model’s lifetime cost is dominated by *inference*, which scales with N at every query. If a model will serve many tokens, it is rational to train a *smaller* model on *more* data than Chinchilla prescribes — paying extra training cost once to save inference cost forever. Sardana & Frankle [4] formalize this by minimizing total (training + expected inference) compute, yielding optimal models that are substantially “over-trained” (token/parameter ratios of hundreds, not 20). This is the explicit rationale behind models like the LLaMA family, trained far past the compute-optimal point for their size.

2.6 Architecture, precision, and hyperparameters

Mixture-of-Experts (MoE). Sparsely-activated models follow their own scaling laws in which the number of experts (or sparsity) is an additional axis. Clark et al. [5] give unified laws for routed models, and Krajewski et al. [6] extend them to fine-grained MoE, showing the compute-optimal granularity and that MoEs can match dense models at lower training *and* inference FLOPs.

Precision. Kumar et al. [7] derive scaling laws in which numerical precision is a first-class variable: training in lower precision effectively reduces the model’s usable parameter count, and there is a precision-aware compute-optimal frontier. This matters as the field moves to FP8 and below.

Hyperparameter transfer (μ P). Yang et al. [8] show that under the *maximal-update parametrization* (μ P), optimal hyperparameters (notably learning rate) become invariant to width, so they can be tuned cheaply on a small model and *transferred zero-shot* to a large one. DeepSeek and others combine this with explicit scaling laws for batch size and learning rate as functions of compute [9].

2.7 From loss to capabilities: “emergence”

Scaling laws are cleanest for *loss*; downstream task accuracy is messier. Some capabilities appear to switch on abruptly at scale (“emergent abilities”). Schaeffer et al. [10] argue much of this apparent sharpness is an artifact of *discontinuous metrics* (e.g. exact-match accuracy): under smooth metrics (per-token likelihood), the same capabilities improve smoothly and predictably. The practical lesson is to forecast with smooth surrogate metrics, not thresholded ones. Caballero et al. [11] further show that some curves have *breaks* — “broken neural scaling laws” — that a single power law misses.

2.8 Caveats

The Chinchilla constants are dataset- and tokenizer-specific, not universal. A replication by Besiroglu et al. [12] recovered the qualitative $a \approx b \approx 0.5$ allocation but found the confidence intervals on the original fit were understated and some constants differ. Power laws also eventually bend: they describe a regime, not an eternal truth, and they say nothing directly about reasoning, factuality, or alignment — only about next-token loss.

3 Reinforcement-learning scaling laws

Modern LLMs are not just pretrained; they are post-trained with RL — first for alignment (RLHF), and more recently with RL on *verifiable rewards* to elicit long-form reasoning (o1, DeepSeek-R1). Scaling behavior here is genuinely different from pretraining, and the science is younger and noisier.

3.1 RLHF and reward-model overoptimization

In RLHF a policy is optimized against a learned *reward model* (RM) that stands in for human preference. Because the RM is an imperfect proxy, pushing the policy too far *Goodharts* the proxy: the RM score keeps rising while true (“gold”) quality peaks and then declines. Gao, Schulman & Hilton [13] measured this precisely. Parametrizing optimization strength by the KL divergence $d = \sqrt{\text{KL}(\pi \parallel \pi_{\text{ref}})}$ between the tuned policy and the reference model, the gold reward follows

$$R_{\text{gold}}(d) \approx d(\alpha - \beta d) \quad (\text{RL}), \quad R_{\text{gold}}(d) \approx d(\alpha - \beta \ln d) \quad (\text{best-of-}n), \quad (3)$$

an initial gain that turns over into decline. Their headline scaling findings: the coefficients α, β vary *smoothly and predictably* with RM size and with the amount of RM training data — **larger reward models are harder to overoptimize** — while, strikingly, the overoptimization behavior was largely *independent of policy size* over the range tested. This gives a principled way to choose a KL budget and to forecast the benefit of a bigger RM.

3.2 RL for reasoning: train-time compute scaling

The o1 / R1 generation post-trains with RL against *automatically checkable* rewards (correct math answer, passing unit test). The central empirical claim, reported for o1, is that reasoning accuracy improves *log-linearly* in **train-time RL compute** — a scaling axis distinct from pretraining compute. DeepSeek-R1 [14] demonstrated the regime openly: starting from a base model, pure RL (their “R1-Zero”) spontaneously grows longer chains of thought and rising benchmark accuracy as RL training proceeds, with no supervised reasoning traces. The response length and the accuracy climb together over RL steps, an emergent “think longer” behavior driven only by the reward signal.

An important precursor outside language is Hilton et al. [15], which established scaling laws for single-agent RL by defining an *intrinsic performance* measure that, unlike raw reward, is power-law in compute; they find an optimal trade-off between environment interaction and model size analogous to the N -vs- D balance in pretraining. Jones [16], on board games, earlier showed clean power laws relating train-time and test-time compute and the ability to *trade one for the other* — the conceptual seed of test-time scaling.

3.3 Test-time (inference) compute scaling

A defining feature of the reasoning regime is that performance also scales with compute spent *at inference* — sampling more, searching, or simply generating longer chains of thought. Snell et al. [17] study how to spend a test-time budget optimally (best-of- N vs. verifier-guided tree search vs. sequential revision) and show the optimum depends on problem difficulty. The provocative headline: on some problem distributions, a *smaller* model given *more* test-time compute beats a *larger* model under a matched total FLOP budget — test-time compute and parameters are partially substitutable, mirroring Jones’s board-game result. The common methods scale differently:

- **Best-of- N / majority vote (self-consistency).** Accuracy rises with N but with diminishing returns, typically saturating as the model’s coverage of correct answers tops out (the pass@ k ceiling).
- **Verifier / process-reward search.** A learned verifier or process-reward model [18] re-ranks or guides search; with a good verifier this scales better than majority vote, because it can select rare correct solutions the majority misses.
- **Long chain-of-thought.** RL-trained models extend their own reasoning at inference; accuracy scales with the number of reasoning tokens, up to a model-dependent limit.

3.4 Allocating compute across the pipeline

The reasoning regime turns scaling into a *three-way* allocation problem — pretraining compute, RL post-training compute, and test-time compute — rather than the single N -vs- D trade-off of pretraining. The emerging (and still unsettled) picture:

- For *easy* queries, extra test-time compute is wasteful; a well-pretrained model answers directly.
- For *hard, verifiable* queries, test-time and RL compute pay off strongly, and can be cheaper than the pretraining compute needed to reach the same accuracy by scale alone.
- Test-time scaling is bounded by the base model’s *coverage*: search and voting can only surface solutions the model can generate with nonzero probability. RL training and pretraining raise that ceiling; inference scaling exploits it.

3.5 Why the RL laws are less settled

Several factors make RL scaling noisier than pretraining scaling:

- **No clean loss.** Reward is non-stationary (the policy moves the data distribution), bounded, and task-specific; there is no universal analogue of cross-entropy that is power-law over many decades.
- **Proxy reward $\neq$ true objective.** Overoptimization (Eq. (3)) means “more RL” is not monotonically good.
- **Elicitation vs. teaching.** It is debated how much RL *adds* new capability versus *surfaces* latent capability from pretraining; the two imply different scaling ceilings.
- **Verifier dependence.** Gains hinge on reward/verifier quality, which itself scales — coupling two scaling problems.

The robust, repeatedly-observed facts are the qualitative ones: train-time RL compute and test-time compute both buy accuracy with diminishing returns; bigger reward models resist Goodharting; and test-time compute partially substitutes for parameters.

4 Synthesis and practical takeaways

	Pretraining	Reinforcement learning
Metric that scales	Cross-entropy loss (clean power law)	Task reward / accuracy (noisy, bounded)
Primary resources	Params N , data D , compute $C = 6ND$	RL train compute, test-time compute, verifier quality
Headline law	$\mathcal{L} = E + A/N^\alpha + B/D^\beta$; $N \propto \sqrt{C}$, $D \propto \sqrt{C}$	Log-linear accuracy in RL compute; turnover in KL (overopt.)
Optimal allocation	≈ 20 tokens/param (Chinchilla); more if inference-heavy	3-way: pretrain vs. RL vs. test-time, difficulty-dependent
Maturity	Mature, predictive, widely replicated	Emerging, qualitative, regime-specific
Key failure mode	Data wall; metric discontinuities	Reward Goodharting; coverage ceiling

Table 1: Pretraining vs. RL scaling at a glance.

Bottom line.

1. *Pretraining is a solved planning problem in principle.* Fit Eq. (2) on small runs, allocate N and D near $D \approx 20N$, and adjust toward over-training if the model will serve heavy inference or if data is constrained (repeat up to ~ 4 epochs nearly for free).
2. *RL adds new, partly-substitutable compute axes.* Train-time RL and test-time search both raise accuracy on verifiable tasks, and a smaller model + more inference can beat a bigger one — but only up to the base model’s coverage ceiling.
3. *Watch the proxy.* RLHF gains are bounded by reward-model error; scale the reward model (not just the policy) to push the overoptimization turnover further out.
4. *Forecast with smooth metrics.* Treat “emergent” jumps skeptically; use likelihood-based surrogates to extrapolate capabilities.

References

- [1] J. Kaplan et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [2] J. Hoffmann et al. *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556, 2022.
- [3] N. Muennighoff et al. *Scaling Data-Constrained Language Models*. arXiv:2305.16264, NeurIPS 2023.
- [4] N. Sardana, J. Frankle et al. *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws*. arXiv:2401.00448, 2024.
- [5] A. Clark et al. *Unified Scaling Laws for Routed Language Models*. arXiv:2202.01169, 2022.
- [6] J. Krajewski et al. *Scaling Laws for Fine-Grained Mixture of Experts*. arXiv:2402.07871, 2024.
- [7] T. Kumar et al. *Scaling Laws for Precision*. arXiv:2411.04330, 2024.
- [8] G. Yang et al. *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer* (μP). arXiv:2203.03466, 2022.
- [9] DeepSeek-AI. *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. arXiv:2401.02954, 2024.
- [10] R. Schaeffer, B. Miranda, S. Koyejo. *Are Emergent Abilities of Large Language Models a Mirage?* arXiv:2304.15004, NeurIPS 2023.
- [11] E. Caballero et al. *Broken Neural Scaling Laws*. arXiv:2210.14891, 2022.
- [12] T. Besiroglu et al. *Chinchilla Scaling: A Replication Attempt*. arXiv:2404.10102, 2024.
- [13] L. Gao, J. Schulman, J. Hilton. *Scaling Laws for Reward Model Overoptimization*. arXiv:2210.10760, ICML 2023.
- [14] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948, 2025.
- [15] J. Hilton et al. *Scaling Laws for Single-Agent Reinforcement Learning*. arXiv:2301.13442, 2023.
- [16] A. L. Jones. *Scaling Scaling Laws with Board Games*. arXiv:2104.03113, 2021.
- [17] C. Snell, J. Lee, K. Xu, A. Kumar. *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters*. arXiv:2408.03314, 2024.
- [18] H. Lightman et al. *Let’s Verify Step by Step*. arXiv:2305.20050, 2023.